ClearAuth AI

Prior Authorisation Screening & Review Platform — Interview Explanation Guide

How to use this guide: Read Section 1 (the 60-second pitch) until you can say it without looking. Then skim Sections 2–15 so you can go deeper when asked. Section 16 has likely interview questions with ready answers, and Section 17 is your **Do's and Don'ts** list. Everything here is taken from the project specification (Clearauth_AI.pdf).

1. The 60-second pitch (say this first)

"ClearAuth AI is an end-to-end **prior authorisation screening and review platform** for a health plan called Brightwater. Before a member gets one of five services — lumbar spine MRI, cervical spine MRI, total knee replacement, bariatric surgery or an in-lab sleep study — the doctor's office must ask the plan for approval.

My system **cleans messy request data** coming from portal, fax and EDI, **reads each service's criteria directly from its own medical-policy PDF** (nothing hard-coded), **screens** every criterion as Met, Not met or Not documented, uses **two ML models** to estimate approval likelihood and hospital length of stay, and an **NLP model** to classify each sentence of the attached clinical documents and flag contradictions.

Then a **LangGraph multi-agent workflow** using Gemini writes a review and a recommended decision — but it **stops at a human-in-the-loop gate**. Only a UM Nurse who did not open the review can decide. After the decision, agents draft the right letter. Everything is stored in an auditable database, served through a **FastAPI** backend and a **Streamlit** dashboard, with role-based access and a RAG chatbot that answers only from the plan's documents."

One-line version

"An AI assistant that prepares prior-authorisation reviews for nurses — it cleans data, checks policy criteria, predicts, reads documents and drafts letters — but a human always makes the final decision."

Core philosophy (repeat this in interviews — it impresses)

- **Tools calculate, LLMs only explain.** Every number, criterion result and figure is computed by plain, testable Python code. The language model only writes prose about facts it is given.
- **Nothing is decided by a model.** The workflow stops before any decision; a human nurse decides.
- **No business rule in the code.** Criteria and thresholds are read from policy PDFs at run time; settings live in `config/config.py`.
- **Everything is traceable and auditable.** Every run, review, decision, LLM call and chat question is stored in a database.

2. The business problem (in simple words)

Prior authorisation = the doctor's office must get the insurer's permission before a costly service is given. Each request comes with a **questionnaire** (answers) and some **clinical documents** (letters, reports). Each of the 5 services has its own **medical policy** listing exact criteria.

Problem	What goes wrong	How ClearAuth AI fixes it
1. Messy data from 3 channels	Heights in feet/inches, weights in stones/pounds, treatment in days, member numbers in 3 formats. Comparing an unconverted value to a criterion gives a wrong result.	A cleaning pipeline converts every answer to one canonical form exactly as the Prior Authorisation Rules say.
2. Look-alike policies	All 5 policies share one template. Lumbar and cervical MRI policies look almost identical but differ in a figure and red flags. Reading the wrong one passes/fails the wrong requests.	Criteria are read only from that service's own PDF; the code checks the policy's service code matches the request.
3. Documents contradict questionnaire	An attached letter may say '16 weeks of physio' while the form says 8. A busy reviewer can miss it.	An NLP model reads every sentence and the tools flag weeks that disagree with the questionnaire.
4. Provider Services calls	Reps may confirm a request only if the caller gives the reference AND the member number, and must never predict approval.	Role-aware chat that follows a strict 'confirmation line' and never predicts outcomes.

The five services

Code	Service	What its criteria cover
RAD-LSM	MRI of the lumbar spine	Minimum age; conservative treatment unless a listed red flag; when a recent scan may be repeated
RAD-CSM	MRI of the cervical spine	Same three, but its own treatment length and its own red flags
ORT-TKR	Total knee replacement	Minimum age, conservative treatment, X-ray grade, knee score, BMI ceiling
BAR-WLS	Bariatric surgery	Minimum age, BMI floor (lower with a listed condition), weight programme, psychological assessment
SLP-PSG	In-laboratory sleep study	Minimum age, sleepiness score, need for the lab (listed condition or inconclusive home test)

The three user roles

Role	What they do	Access
Intake Coordinator	Opens (starts) a review of a request	Intake, review, audit, chat — NOT decision
UM Nurse Reviewer	Decides a review — but never one they opened	Every area
Provider Services Representative	Answers phone calls from doctors' offices	Chat only

Separation of duties: the person who opens a review cannot decide it. This comes from the Utilisation Management Policy and is the reason the workflow pauses at a 'gate'.

3. Architecture at a glance

Think of the system as layers, each building on the previous one:

#	Layer	Folder	Job in one line
1	Data preparation	data_pipeline/	Clean 5 raw CSVs, write a data-quality report
2	Modelling	modeling/	Train approval classifier, stay regressor, evidence NLP classifier
3	Knowledge base (RAG)	knowledge/	Read 16 PDFs, split into sections and chunks, store in ChromaDB, read policy criteria

#	Layer	Folder	Job in one line
4	Agent tools	tools/	Deterministic code: screening, evidence, predictions, retrieval, letters, decision checks
5	Persistence	persistence/	SQLAlchemy models and repository for 6 audit tables
6	Agentic workflow	agentic/	LangGraph: 10 nodes, 7 agents, human gate, checkpoint
7	REST service	api/	FastAPI endpoints with Pydantic schemas
8	Access control	access/	Password hashing, session tokens, role-based area checks
9	Chat assistant	chat/	RAG chatbot + role-aware request status answers
10	Dashboard	dashboard/	Streamlit UI: sign-in + 5 pages

Tech stack

Area	Technology
Language / data	Python, pandas
Machine learning	scikit-learn Pipelines (forest models for tabular data — the config holds tree counts and leaf minimums; a text classifier for evidence)
Vector DB / RAG	ChromaDB (persistent), sentence-embedding model named in config, PDF text extraction
LLM	Google Gemini via the `google.genai` SDK; key and model name read from `.env`
Agents	LangGraph (StateGraph, conditional edges, interrupt + checkpointer saved to `checkpoint.db`)
Database	SQLAlchemy ORM over `database/clearauth.db`
API	FastAPI + Pydantic, served by Uvicorn on port **8081**
UI	Streamlit on port **8080**
Security	Salted password key-derivation (hex), random session tokens, constant-time comparison

End-to-end flow (the story of one request)

Step	What happens
1	Request arrives (portal/fax/EDI) → cleaned into canonical form; only the latest submission is kept.
2	Its own medical policy PDF is read → list of criteria with their figures.
3	Screening: each criterion → Met / Not met / Not documented → overall screening + decisions open.
4	Approval model → likelihood + band (High / Medium / Low). Stay model (inpatient surgery only) → expected nights.
5	Evidence model reads every document sentence → evidence type → linked criteria → disagreement flags.
6	Intake Coordinator opens review → 5 agents (planner, criteria, evidence, predictions, writer) → review saved as AWAITING.
7	Workflow PAUSES at the gate (checkpointed to disk — survives a restart).
8	A different UM Nurse decides → decision validated → workflow resumes.
9	Approve → provider letter + member notice. Request info → information-request letter. Refer → referral summary (no letter).
10	Everything recorded; the run is closed as COMPLETE.

4. The datasets

File	What it is	Used for	Rows (received → kept)
authorisation_history_train.csv	Past requests with decision (Approved/Denied); 44 columns	Train approval model	31,751 → 29,842
authorisation_history_eval.csv	Later past requests	Test approval model (never fitted on)	7,926 → 7,467
surgical_stays_train.csv	Completed inpatient stays (knee / bariatric); 21 columns	Train stay model	30,474 → 29,782
surgical_stays_eval.csv	Later stays	Test stay model	7,610 → 7,436
clinical_note_sentences.csv	One labelled sentence per row; 8 columns; 9 evidence types	Train evidence NLP model	36,651 → 35,695
data/input/open_requests.csv	The live queue (first 40 columns, no answer)	Screening and review	40 requests after cleaning
data/input/request_notes.csv	Documents sent with live requests	Evidence reading	102 documents

Why train/eval split by time? The eval files contain requests that came after the training ones — this mimics real life (predict the future from the past) and avoids leakage.

Leakage to remember: columns like `information_requested`, `reviewer_level`, `decision_turnaround_days`, complication/readmission are known only after the decision or stay, so they should not be model inputs. Member sex and region are recorded for fairness checks only.

Data-quality defects and how they are fixed

Defect	Example	Fix
Extra spaces in names	' Larchmont Community Hospital '	Collapse whitespace runs to one space
Missing written many ways	N/A, NA, -, unknown, Not recorded	Only config NULL_TOKENS → NaN. 'None' (no conditions) is kept as a real answer
Duplicate rows	Same row twice	Keep the first copy
Resubmissions	Same reference, higher submission_sequence	Keep only the highest sequence (latest), even if older one had a decision
Spelling variants	EDI / Facsimile / Expedited / Fri	Mapping dictionaries → one canonical value
Yes/No variants	Y, True, NO, n, false	Boolean mapping
Identifier formats	8276601, BWH3060498, bwh-2797357, G4730	Canonical BWH-1234567, PA-123456, ICD code dot after 3rd char (G47.30)
Numbers with units	3 days, 14 mo, 83.9%, 34.1 kg/m2	Regex patterns read the number
Scores with scale	KL grade 3, 13/48, 16/24	Read the score; beyond scale → missing (row kept)
Treatment in days	50 days, 31 wks	Days ÷ 7, remainder dropped (50 days = 7 weeks)
ASA class 3 ways	ASA III, IV, ASA 2	Roman/number → integer; 'ASA V' refused
Height / weight notations	5 ft 6 in, 1.72 m; 13 st 8 lb, 180 lb	Exact conversion to cm / kg, rounded once, half-up
Free-text conditions	Asthma, HTN, COPD, high blood pressure	Canonical names, de-duplicated, sorted, joined
Impossible readings	Knee score 50, BMI 207, stay of 0 nights	Set to missing (row kept); stay with no valid target removed
HTML in text	, , -	Strip markup, unescape entities, normalise spaces
All-caps sentences	THERE ARE NO FEATURES...	Lower-case only fully-capital text

Defect	Example	Fix
Too-short sentences / stale length column	1–2 words; length measured on raw text	Drop short ones; recompute length on clean text

Key principle: an unknown format is **refused with a ValueError** naming the column and values — never guessed. All five datasets are cleaned before any is saved, so a failure never leaves half-cleaned files. A JSON **data-quality report** proves each cleaning step (rows received, kept, removed, duplicates, missing cells, notation counts, target shares), and the removal reasons add up exactly to rows_removed.

The worked example (great to quote!)

Section 12 of the Prior Authorisation Rules: a fax for a test policy — minimum age 21, 8 weeks of treatment unless a red flag, BMI below 38.0.
- **52 days** → 52 ÷ 7 = **7 whole weeks** (remainder dropped) → treatment **Not met**.
- Red flag written **'Nil'** → no red flag.
- **5 ft 10 in = 177.8 cm**, **15 st 7 lb = 98.4 kg** → BMI = 98.4 / 1.778² = **31.1** (below 38 → Met).
- Result: **REFER TO PHYSICIAN**.
- With 56 days (= 8 weeks) every criterion is Met → APPROVE. With the red flag blank → REQUEST MORE INFORMATION.

5. Reading the policy and screening a request

Criteria are NOT in the code

`knowledge/policy_reading.py → read_medical_policy(service_code)` opens the service's own PDF, checks it states its own service code once, finds the Section 4 'coverage criteria', checks criteria are numbered 1, 2, 3… in order, and matches each criterion sentence against fixed sentence patterns (regex). It extracts figures (ages, weeks, BMI as exact decimal), red-flag lists (and verifies the count word, e.g. 'three red flags', matches the list) and listed conditions. If a policy of another service is passed, screening refuses it.

Three results per criterion

- **Met** — minimum/maximum met at the figure itself; a ceiling written as 'below' is met only under it.
- **Not met** — the answer is known and fails.
- **Not documented** — an answer needed is blank; the system lists which questionnaire answers are missing.
- Two-way criteria (e.g. treatment or red flag): Met if either way is met; Not met only if both ways are known to fail.
- Follow-on questions (e.g. months since last scan) are not needed if the first answer settles it.
- BMI = kg / m², computed exactly, rounded once half-up to 1 decimal; if height or weight is blank → Not documented.

Screening outcome and decisions open

Criteria results	Screening	Decisions the nurse may take
Any criterion Not met	REFER TO PHYSICIAN	Refer only
None Not met, at least one Not documented	REQUEST MORE INFORMATION	Request information or Refer
Every criterion Met	APPROVE	Approve or Refer

Important: the system **never denies**. A nurse can approve or ask for information; anything doubtful goes to a physician.

6. The three machine-learning models

Model	Type	Learns from	Metric & minimum	Output
Approval model	Binary classifier (forest)	History train + 3 criteria counts (Met / Not met / Not documented)	Held-out ROC AUC ≥ config minimum; calibrated (mean score within 0.03 of real approval share)	Approval likelihood + band High / Medium / Low
Stay model	Regressor (forest)	Surgical stays train	Held-out R^2 ≥ minimum; MAE < 0.75 × baseline MAE (always predicting the mean)	Expected nights in hospital (inpatient only)
Evidence model	NLP text classifier (vectoriser + estimator in one pipeline)	Clinical note sentences (stratified split)	Held-out macro F1 ≥ minimum; ≥ 75% recall per class	One of 9 evidence types per sentence

Points worth explaining

- **Pipelines:** each model is a scikit-learn Pipeline — imputation (fill blanks the same way as training), one-hot encoding for categories, then the estimator as the last named step. So the saved model can score one raw row directly and there is no train/serve skew.
- **Feature schema** (column names in order) is saved as JSON and checked at prediction time.
- **Sanity / behaviour tests (monotonicity):** turning a criterion from Not met → Met must raise the approval score for ≥95% of affected requests; Not documented → Met for ≥80%; average rise ≥0.1. For stay: ASA 4 vs 2 adds ≥1 night on average; bilateral knee adds ≥1 night; no enhanced-recovery programme adds ≥0.5 night.
- **Generalisation gap** (train score − held-out score) is reported to show learning vs memorising.
- **Why macro F1** for evidence: classes are imbalanced; macro F1 treats rare evidence types equally.
- **Why ROC AUC + calibration**: we show a probability to people, so ranking quality and honest probabilities both matter.
- If any model misses its minimum, training stops and **nothing is saved**.
- The predictions **decide nothing** and are never told to an office or member.
- Longer stay expected = expected nights more than the configured margin above the inpatient days requested.

The 9 evidence types (from the evidence guides)

Conservative treatment, Failed medication, Red-flag symptom, Functional limitation, Prior investigation, Weight-management programme, Other condition, Ruled out — plus **Background** (bears on no criterion).

7. Knowledge base (RAG) — 16 documents

Documents: 5 medical policies, Utilisation Management Policy, Prior Authorisation Rules, Past-Reviews Casebook (indexed but **excluded from chat** because it names members), and 8 evidence guides.

Step	Module	What it does
Extract	pdf_extraction.py	Read every page's text in order; refuse empty PDFs
Section split	chunk_splitting.py	Remove the one header line; split at numbered headings 1, 2, 3… (145 sections)
Chunk	chunk_splitting.py	Whole sentences only, max size in characters, with overlap; never crosses sections/documents; deterministic chunk_id
Embed & store	vector_index.py	Persistent ChromaDB collection, embedding model from config, rebuilt (not appended) each time

Step	Module	What it does
Retrieve a section	find_section_passage	Search only ONE document and verify the passage starts with the expected heading
Chat search	search_documents	Multiple queries at once, top-k with distances, casebook filtered out

Why section-aware chunking? So that a criterion is never cut in half and lumbar/cervical passages never mix. The indexing run prints each policy's 'documentation to send' to prove each service gets its own passage.

8. Reading clinical documents and flagging disagreements

- Each document is split into sentences (in order) and each sentence gets an evidence type from the NLP model.
- Each type maps to criterion kinds → the sentence records which of the request's own criteria it bears on.
- For **conservative-treatment** sentences, the number of weeks is extracted — digits ('16 weeks'), decimals, words ('eight weeks'), compounds ('twenty four weeks'). 'Six weeks ago' is a time, not a duration, so it is ignored.
- Flag **DISAGREES_WITH_QUESTIONNAIRE** if stated weeks ≠ questionnaire weeks; **DESCRIBES_BLANK_ANSWER** if the questionnaire left weeks blank.
- **A flag never changes an answer** — documents are evidence, not answers. The nurse decides.

9. The multi-agent workflow (LangGraph)

Each agent = **exactly one LLM call** that interprets facts the tools already established. Replies use a fixed format of **labelled fields** (e.g. PRIORITY:, CHECKS:) so they can be parsed reliably; a robust parser handles bold markers, decoration and repeated labels, and refuses a reply with missing fields.

#	Node	Agent / job	Output fields
1	load_request_context	No LLM. Gathers request, documents, screening, evidence, disagreements, likelihood, stay; opens run (RUNNING)	run_id, review_id, context
2	plan_review	Planner agent — what must the review establish?	PRIORITY, CHECKS, RATIONALE
3	assess_criteria	Criteria analyst — explains Met / Not met / Not documented	CRITERIA MET, CRITERIA NOT MET, NOT DOCUMENTED, CRITERIA SUMMARY
4	review_evidence	Evidence analyst — with evidence-guide checks from RAG	EVIDENCE FOUND, DISAGREEMENTS, CHECKS TO MAKE, EVIDENCE SUMMARY
5	weigh_predictions	Likelihood analyst — explains the two model figures	LIKELIHOOD, EXPECTED STAY, PREDICTION SUMMARY
6	compose_review_recommendation	Writer — review + one recommended decision (must be an open decision); saves review AWAITING	RECOMMENDATION, RECOMMENDED DECISION
7	request_review_decision	**THE GATE** — interrupt; wait for nurse; validate decision	review_decision
8	draft_office_letter	Letter agent (Approve / Request info)	SUBJECT, LETTER
9	draft_member_notice	Notice agent (Approve only, plain words)	HEADLINE, NOTICE

#	Node	Agent / job	Output fields
10	record_review_outcome	No LLM. Saves decision, letters, referral summary; run COMPLETE	—

Branching (two conditional edges, both by route_review_decision)

- **APPROVE** → draft_office_letter → draft_member_notice → record (5 + 2 = 7 LLM calls)
- **REQUEST MORE INFORMATION** → draft_office_letter → record (6 calls)
- **REFER TO PHYSICIAN** → record directly with a referral summary (5 calls)

Human-in-the-loop and fault tolerance

- The run pauses with LangGraph's interrupt; state is saved by a **checkpointer to checkpoint.db**, so it can be resumed by another process even after the machine restarts.
- **find_decision_refusal** checks, in order: missing fields → unknown decision → role not UM Nurse → same person who opened it → decision not open for this screening → approval letter would lack site of service / inpatient days. Refused before any LLM call is spent.
- State key `llm_call_count` uses an additive reducer; other keys are replaced.
- Letters are validated: subject must name reference and service; no forbidden terms (e.g. model likelihood); no template placeholders like [ ]; must end with the fixed sign-off. Member notice carries no clinical detail beyond the service name. Letter details (reference, member number, site, policy) are filled by tools, **never by the LLM**.

10. Persistence — six audit tables

Table	Holds
user_accounts	Username, display name, role, salted password hash, salt, current session token
agent_runs	run_id, thread_id, request, coordinator, status (running / awaiting decision / complete / failed), call count, timestamps
authorisation_reviews	The stored review: tool results, each agent's write-up, recommendation, status, letters, notice, referral summary
review_decisions	Decision, decided_by, role, reason — its own row because auditors ask about it
llm_calls	Every LLM call: agent, model, prompt/response characters, latency in ms
chat_turns	Every chat question and answer, including refusals

Single engine + session factory (thread-safe). A **repository pattern**: only repository.py builds queries; callers own the transaction so related writes commit together (e.g. review + LLM call + run status in one transaction).

11. REST API (FastAPI, port 8081)

Method & path	Purpose	Notes
POST /auth/signin	Username + password → session token	Wrong user and wrong password give the same 401
POST /auth/signout	End session	Token cleared
GET /queue?service_code=	Ranked queue (urgent first, then by reference)	No LLM
GET /queue/{ref}	Full request detail before spending any LLM call	404 if unknown
POST /reviews	Run the 5 agents to the gate	201; 409 if already reviewed; per-request lock
GET /reviews?status_value=	List reviews by status	Newest first

Method & path	Purpose	Notes
POST /reviews/{id}/decision	Validate decision and resume workflow	409 if already decided / refused
GET /reviews/{id}/calls	Call budget: calls made vs expected, latency	—
POST /chat/ask	Chat question (1 LLM call)	Member numbers masked first
GET /chat/turns	Own conversation	—
GET /audit/chat-turns	Everyone's recent chat turns	Audit

- Pydantic schemas validate input → **422** before any work. Status codes: 200, 201, 401 (not signed in), 403 (role not allowed), 404, 409 (conflict), 502 (any failure — one global handler marks the run FAILED).
- Queue and request details are computed once at start-up from the same tools the workflow uses → queue, detail and stored review always agree.
- Only 3 endpoints may cost LLM calls: POST /reviews, POST /reviews/{id}/decision, POST /chat/ask.

12. Access control and security

- Passwords stored as a salted key-derivation hash (algorithm and rounds from config) — the database never holds a password.
- Each sign-in issues a fresh random token (one session per account) sent in a header.
- **authorize_request** is attached to the whole guarded router, so a new endpoint cannot forget the guard. It maps method + route template to an area and checks the role's allowed areas → 403 even if the resource doesn't exist.
- The **server** enforces access, not the dashboard (the dashboard only hides pages for convenience).
- Member numbers never reach the LLM or the chat record.

13. The authorisation chat assistant (RAG)

- One question = one LLM call. Answers **only from the plan's documents**; question classified into 6 types (e.g. medical policy, evidence guidance, request status, out of scope).
- Two searches (current question + previous question for context), passages within a relevance distance threshold; otherwise a polite refusal.
- Remembers the user's own last few turns (conversation memory); record-based answers are replaced in history so review data never reaches a prompt.
- Questions about a specific request are answered by the **service from the database**, not by the LLM.

The 'confirmation line' for Provider Services

Rep asks…	Rep hears
Will it be approved? / How likely?	A fixed clinical-determination line — nothing more, even with reference + member number
About a request without naming it	'Request not named' message
With correct reference AND matching member number	Service and status (+ answers asked for, if information was requested)
Anything else (no / wrong member number, not open)	A 'no confirmation' message that never confirms the request exists

Coordinators and Nurses hear the review's full record (screening, figures, evidence counts, decision).

14. The dashboard (Streamlit, port 8080)

Page	What it shows
Sign in	Username / password; session-ended message
Review queue	Counts (total, urgent, all met, missing answers, not met); filter by service; table with criteria counts, disagreements, screening, likelihood, band
Request detail	Headline, screening graph (criterion → screening → decisions), evidence tab with chart, questionnaire tab, review tab to start a review
Decision queue	Reviews awaiting decision; recommendation, criteria, disagreements; form offers only open decisions; blocks own review
Audit trail	Reviews by status, decision and who took it, agent write-ups, letters, call budget and latency; chat log
Chat	Own conversation with type badges and sources

All requests go through one `api_client.call_service` function; the dashboard holds **no business rule**.

15. How to run it (6 entry points)

Step	Command	Result
0	pip install -r requirements.txt	Dependencies
1	python run_database_setup.py	clearauth.db, 6 tables, 14 seed accounts (safe to re-run: 0 created, 14 present)
2	python run_data_pipeline.py	5 cleaned CSVs in processed/ + output/data_quality_report.json
3	python run_model_training.py	3 models + 2 schemas in output/models/ + model_metrics.json
4	python run_knowledge_indexing.py	16 docs, 145 sections, chunks in database/chroma_store
5	python run_api_service.py	API on :8081 (docs at /docs); checks required files first
6	python -m streamlit run run_dashboard.py	Dashboard on :8080

16. Likely interview questions — with ready answers

Q. Why not let the LLM decide approval?

A. Healthcare decisions need accountability and must follow exact policy rules. LLMs can hallucinate and are not reproducible. So deterministic code computes every result, the LLM only explains it, and a qualified nurse makes the final call.

Q. Why read criteria from PDFs instead of hard-coding?

A. Policies change. If thresholds are in code, every policy update needs a code change and risks mismatches (e.g. using lumbar figures for cervical). Reading them at run time keeps one source of truth.

Q. How did you prevent hallucinations?

A. Agents only receive facts computed by tools; prompts say 'use only what you are given'; replies are parsed into labelled fields and validated (recommended decision must be an open decision, letters cannot contain forbidden terms or placeholders); letter details are filled by code, not the LLM; the chat refuses when retrieval distance is too large.

Q. What is human-in-the-loop here?

A. The LangGraph workflow interrupts at the decision gate, saves state to a SQLite checkpoint, and resumes only when a UM Nurse who did not open the review submits a valid decision.

Q. How does the workflow survive a restart?

A. LangGraph checkpointer writes the paused state to checkpoint.db; the run is resumed using its thread_id from any process.

Q. Why ROC AUC and calibration for the approval model?

A. AUC measures ranking ability irrespective of threshold; calibration ensures a 0.7 really means about 70% — important because humans read the number.

Q. Why macro F1 for the evidence model?

A. Classes are imbalanced; macro F1 averages per-class F1 so rare evidence types count equally.

Q. How did you handle missing values?

A. Only explicit null tokens become NaN; 'None' is a valid answer. Impossible readings become missing but the row is kept. In models, a pipeline imputer fills numbers and categories the same way at training and prediction.

Q. How do you avoid data leakage?

A. Time-based eval files, models fitted only on training rows, post-decision columns not used as inputs, and the whole preprocessing inside the pipeline.

Q. What is the role of RAG?

A. To ground every statement in Brightwater's documents — to read a policy's 'documentation to send', an evidence guide's reviewer checks, and to answer chat questions with sources.

Q. How did you chunk documents?

A. First split by numbered sections, then into whole-sentence chunks with overlap, never crossing a section — so a criterion is never split and passages stay on one subject.

Q. How is security handled?

A. Salted password hashing, random session tokens, constant-time compare, server-side role checks on every guarded route, member-number masking before the LLM.

Q. How do you control LLM cost?

A. Fixed number of calls per run (5 before the gate, 5/6/7 in total by decision); every call logged with latency; a call-budget endpoint compares made vs expected; decisions validated before spending calls.

Q. What was the hardest part?

A. Pick one honestly — e.g. writing cleaning functions that convert every notation exactly (stones/pounds, days to weeks) and refuse unknown formats; or making the agent outputs parse reliably; or the checkpoint/resume logic.

Q. What would you improve?

A. Add model monitoring and drift detection, fairness checks using member sex/region, more automated tests, Docker/Kubernetes deployment, and SHAP explanations for the approval figure.

17. Interview Do's and Don'ts

DO	DON'T
• Start with the 60-second pitch, then let the interviewer choose where to go deeper. • Explain the **business problem first**, then the tech. Use the phrase 'human-in-the-loop'. • Stress the design principle: **tools calculate, LLMs explain, humans decide**. • Use the worked example (52 days → 7 weeks → Refer) to show precise thinking. • Name the stack clearly: pandas, scikit-learn, ChromaDB, LangGraph, Gemini, SQLAlchemy, FastAPI, Streamlit. • Mention metrics with the reason you chose them (ROC AUC + calibration, R^2 + MAE vs baseline, macro F1). • Talk about auditability, security and cost control — these show production thinking. • Be honest about which parts you built personally and which were provided (e.g. config.py was supplied). • Draw the flow on a whiteboard if allowed: Data → Screening → Models → Agents → Gate → Letters → DB → API → UI. • Keep answers to 1–2 minutes and ask 'Would you like me to go deeper into X?'	• Don't say 'the AI approves or denies requests' — it only **recommends**; the system never denies at all. • Don't claim exact scores (AUC, R^2, F1) unless you remember your own run's numbers — say 'it met the configured minimum'. • Don't invent technologies you didn't use (e.g. OpenAI, Pinecone, React) — stay with the real stack. • Don't read out function signatures or config names one by one; explain the idea. • Don't share real member data or claim it is production data — it is a training/assessment project with synthetic data. • Don't get lost in the cleaning details at the start — keep them for when you're asked. • Don't say the LLM 'calculates BMI' or 'reads the criteria' — code does that. • Don't bluff. If you don't know, say how you would find out. • Don't forget the 'why' behind every design choice.

18. Quick-recall cheat sheet

Fact	Value
Services / policies	5 (RAD-LSM, RAD-CSM, ORT-TKR, BAR-WLS, SLP-PSG)
Documents in knowledge base / sections	16 / 145
Raw datasets cleaned	5 (2 history, 2 stays, 1 sentences)
Live open requests / documents	40 / 102
Models	3 (approval classifier, stay regressor, evidence NLP)
Evidence types	9
Workflow nodes / agents	10 nodes / 7 agents (5 before gate + letter + notice)
LLM calls	5 before gate; total 7 approve, 6 request info, 5 refer

Fact	Value
Database tables	6
Roles / seed accounts	3 / 14
Screening outcomes	APPROVE, REQUEST MORE INFORMATION, REFER TO PHYSICIAN
Ports	API 8081, Dashboard 8080

Closing line for the interview: “The main thing I learned is how to combine classic ML, RAG and agents safely — keeping the AI as an assistant that is grounded, auditable and always under human control.”